

# Autopilot — Brief & Build Plan

Sep 26, 2026 · @Kavishani

Autopilot is incident response for revenue: an agent that notices where a store is quietly losing money, fixes it, and proves the fix worked.

## Brief

**One-liner.** Sentry for revenue: Autopilot notices where a store or marketplace listing is quietly losing money, fixes it, and proves the fix worked.

**Problem.** Small merchants cannot watch analytics all day. A broken product page, a mispriced item or a best-seller about to run out can cost hundreds of pounds before anyone notices. Analytics tools show charts; nobody acts on them.

**Solution.** An agent that watches the store 24/7, detects where money is leaking, diagnoses why, and ships a fix to Shopify with the merchant’s approval where needed. Every fix is then tested against a fixed checklist of known buying blockers (missing size guide, no materials, unclear returns, price above market, weak lead image), and rolled back if it does not help.

**Who pays.** Independent Shopify merchants, on a monthly subscription or a share of verified recovered revenue. Bigger prize: marketplaces like Fleek running it across every seller’s listings, because better listings mean more sales and more commission.

**Why it wins.** Most teams will build buyer-side shopping bots. Autopilot is merchant-side, closes the full loop, and puts a money number on screen. Unlike Shopify Sidekick, which waits for you to ask, Autopilot acts on its own and proves its fixes worked.

| Judging criterion          | How Autopilot scores                                                    |
|----------------------------|-------------------------------------------------------------------------|
| Problem / Product thinking | Lost revenue is universal and measurable in £                           |
| Execution                  | Rule-based detection is reliable; one full loop on a real Shopify store |
| AI leverage & autonomy     | Agent detects, diagnoses, fixes and verifies, with set autonomy levels  |
| Innovation / Originality   | Incident response for commerce, tested with AI shopper traffic          |

| Judging criterion   | How Autopilot scores                                       |
|---------------------|------------------------------------------------------------|
| Usefulness / Impact | Every merchant judge wants it for their own store          |
| UX / Experience     | Alerts and approvals on WhatsApp, one tap to fix           |
| Commerce depth      | Real Shopify writes, market price comps, stock velocity    |
| Demo quality        | Break a page live, watch it get caught, fixed and verified |

## Judge questions and answers

Expect these at the table visit and after the demo. Answer in one or two sentences, then point at the screen.

| Likely question                                | Answer                                                                                                                                                                                                                                  |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "Isn't this just AI grading AI?"               | No. Shoppers score pages against a fixed checklist of known buying blockers written before the hack, and the fixer agent never sees it. In production, fixes are verified on real traffic in PostHog; personas are the pre-flight test. |
| "Your break is staged. Of course it recovers." | We break it the way merchants actually do: removing the size info, or pricing 25% above market. Subtle enough that the owner would not notice, which is exactly the point.                                                              |
| "How is this different from Shopify Sidekick?" | Sidekick waits for you to ask. Autopilot notices on its own, runs around the clock, and proves each fix worked or rolls it back. It also works for marketplace sellers, not only Shopify stores.                                        |
| "Where does the £ figure come from?"           | Missing carts per day × average order value × checkout rate. In production that uses real PostHog numbers.                                                                                                                              |

| Likely question                           | Answer                                                                                                                                                                                                  |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "Do AI shoppers behave like real people?" | They do not need to predict every human. Like automated tests for code, they catch known blockers before real customers hit them.                                                                       |
| "What if the agent makes a bad change?"   | Merchants set autonomy levels, prices never go below the margin floor, price changes need approval, every change is stored, and anything that does not improve conversion is rolled back automatically. |
| "Who pays, and how much?"                 | Merchants on a monthly plan or a share of recovered revenue. Marketplaces like Fleek license it across all sellers, because it lifts sales and commission.                                              |
| "Why won't Shopify just build this?"      | They may for their own stores. Our wedge is marketplaces and multi-channel sellers, plus a growing record of which fixes actually work.                                                                 |
| "Why do you need agents, not rules?"      | Detection is rules, for reliability. Working out why a page fails and writing the right fix needs reasoning across the page, the data and the market, which is where the agent earns its place.         |
| "What actually works right now?"          | The full loop on a real Shopify dev store: detect, diagnose, WhatsApp approval, Shopify write, re-test, verified or rolled back.                                                                        |

## How it works

The product is one loop that runs continuously: detect, diagnose, fix, verify, and roll back if the fix did not help.

1. **Detect.** Rules in Supabase scan shopper events every 30 seconds. A rule firing opens an incident with an estimated £-per-day loss.
2. **Diagnose.** Grok reads the product page, product data, recent events and market comps, and explains the likely cause in one sentence.
3. **Fix.** Grok proposes a concrete change (new copy, new price, reorder draft). Depending on the autonomy level, it applies it directly or asks on WhatsApp.

- 4. **Verify.** The shopper agents re-run against the updated page. If conversion recovers, the incident closes as "verified fixed".
- 5. **Roll back.** If conversion does not improve, the agent restores the previous version and flags the incident for the merchant.

**Autonomy levels.** The merchant chooses what the agent may do alone:

| Fix type                                            | Default                    |
|-----------------------------------------------------|----------------------------|
| Rewrite title, description, fill missing attributes | Automatic                  |
| Price change within margin guardrail                | Needs WhatsApp approval    |
| Supplier reorder message                            | Draft only, merchant sends |

**AI shopper traffic.** There is no real traffic at a hackathon, and scripted numbers look staged. Instead, 20–50 Grok shopper personas (budget student, gift buyer, sizing-anxious, brand hunter) read the real Shopify product page and decide whether to view, add to cart or leave. Each decision is scored against a fixed blocker checklist (size guide, materials, returns, price vs comps, lead image) that the fixer agent never sees, so the test stays independent. In production, the same loop runs on real PostHog traffic and the personas become pre-flight tests.

## Architecture

A Next.js app on Vercel with Supabase as the backbone; detection is plain SQL, and Grok is used only for reasoning and writing. That split keeps the demo reliable.

| Partner           | Job in Autopilot                                                                  |
|-------------------|-----------------------------------------------------------------------------------|
| Grok Bot          | Shopper personas; diagnosis; writing fixes                                        |
| Supabase          | Events, incidents, fix history; realtime drives the live feed; cron for detectors |
| Shopify Admin API | Reads products and inventory; applies fixes (product update, variant price)       |
| Tavily            | Market price comps for the pricing detector                                       |
| Wassist           | WhatsApp alerts with Approve / Reject / Why                                       |

| Partner | Job in Autopilot                                           |
|---------|------------------------------------------------------------|
| PostHog | Funnel and before/after charts in the dashboard            |
| Vercel  | Hosts the dashboard and API routes; public link for judges |
| Cursor  | How the team builds it                                     |

Send each shopper event to both Supabase and PostHog. PostHog ingestion can lag by minutes, so detection reads from Supabase for real-time.

### Data model (Supabase)

| Table     | Key columns                                                                                        |
|-----------|----------------------------------------------------------------------------------------------------|
| products  | shopify_id, title, price, cost, inventory, snapshot_json                                           |
| events    | product_id, persona, type (view / add_to_cart / leave), reason, created_at                         |
| incidents | product_id, detector, loss_per_day_gbp, diagnosis, status (open / fixing / verified / rolled_back) |
| fixes     | incident_id, before_json, after_json, autonomy, approved_by, applied_at                            |
| settings  | autonomy per fix type, margin floor %                                                              |

### The three detectors

| Detector             | Rule                                                               | Loss estimate                                                | Fix                                                         |
|----------------------|--------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------------------|
| 1. Views, no carts   | Add-to-cart rate under 30% of store average over the last 20 views | (expected carts – actual) × avg order value, scaled to a day | Rewrite copy, fill missing size or material, reorder images |
| 2. Price out of line | Price over 20% above the median of Tavily comps                    | Lost carts attributed to "too expensive" persona reasons     | New price, never below the margin floor; needs approval     |

| Detector          | Rule                                                                    | Loss estimate                                        | Fix                                  |
|-------------------|-------------------------------------------------------------------------|------------------------------------------------------|--------------------------------------|
| 3. Stock-out risk | Days of stock left (inventory ÷ daily sales) under 3 on a top-5 product | Daily revenue of that product × days it would be out | Drafted supplier reorder on WhatsApp |

Build detector 1 first. It is the heart of the demo; 2 and 3 are stretch.

## Build plan

The one rule: detector 1 with the full loop working by 15:00 beats three half-working detectors.

### Phase 1 · until 11:45 — Foundations

- ☐ Next.js repo deployed to Vercel (empty page live)
- ☐ Supabase project, tables from the data model, env vars on Vercel
- ☐ Shopify dev store seeded with 8–10 products with good copy, prices and stock
- ☐ Shopify Admin API token; script that syncs products into Supabase
- ☐ Join the wassist channel on Discord and get WhatsApp sending a test message

### Phase 2 · until 13:30 — Shoppers and detection

- ☐ Persona shopper: Grok reads a product page and returns view / add\_to\_cart / leave + a one-line reason as JSON
- ☐ Runner that sends 20 personas across the store and writes events to Supabase and PostHog
- ☐ Detector 1 as a SQL query; opens an incident with a £-per-day estimate
- ☐ Fixed blocker checklist written and locked; "Break" button that makes a realistic mistake via the Shopify API (removes size info, or prices 25% above market)

### Phase 3 · until 15:00 — Fix and verify (the core)

- ☐ Diagnosis prompt: page + events + persona reasons → one-sentence cause
- ☐ Fix prompt: returns new title and description; applied through Shopify API, before/after stored in fixes
- ☐ WhatsApp alert with Approve / Reject / Why; approval triggers the fix
- ☐ Verify: re-run personas on the fixed product, compare conversion, close as verified or roll back

- ☐ Run the full loop 3 times in a row without touching code

**Phase 4 · until 16:00 — Dashboard and stretch**

- ☐ Dashboard: live incident feed (Supabase realtime), revenue-at-risk counter, before/after conversion
- ☐ PostHog funnel chart embedded or linked
- ☐ Stretch: detector 2 (price vs Tavily comps) with approval flow
- ☐ Stretch: detector 3 (stock-out risk) with drafted reorder

**Phase 5 · 16:00–16:30 — Ship**

- ☐ Feature freeze at 16:00, final deploy
- ☐ Record a 3-minute backup video of the working loop
- ☐ Submit with the Vercel link
- ☐ Rehearse the demo twice with a timer

**Team split**

| Team size | Person A                                       | Person B                      | Person C                            |
|-----------|------------------------------------------------|-------------------------------|-------------------------------------|
| Solo      | Everything, in phase order; skip detectors 2–3 | —                             | —                                   |
| 2 people  | Shopify, detectors, fix and verify             | Personas, WhatsApp, dashboard | —                                   |
| 3 people  | Shopify sync, detectors, fix and verify        | Personas, prompts, diagnosis  | WhatsApp, dashboard, demo and pitch |

**Demo script (3 minutes)**

| Time      | What happens on screen   | What you say                                                                       |
|-----------|--------------------------|------------------------------------------------------------------------------------|
| 0:00–0:20 | Title slide or dashboard | "Small merchants lose money silently. Autopilot is incident response for revenue." |

| Time      | What happens on screen                                                           | What you say                                                                                                           |
|-----------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 0:20–0:50 | Healthy store; persona shoppers streaming into the live feed                     | "These are AI shoppers reading our real Shopify store and deciding whether to buy."                                    |
| 0:50–1:30 | Press Break (size info removed); conversion drops; incident fires with £ per day | "I made the kind of mistake merchants make every day. Autopilot caught it and costed it: missing carts × order value." |
| 1:30–2:10 | Diagnosis appears; phone buzzes; approve on WhatsApp; page updates on Shopify    | "It explains why, proposes a fix, and asks me. One tap."                                                               |
| 2:10–2:45 | Shoppers re-run; conversion recovers; incident closes as verified                | "It doesn't just fix. It proves the fix worked, or rolls it back."                                                     |
| 2:45–3:00 | Business model line                                                              | "Subscription, or a share of revenue we verifiably recover."                                                           |

## Risks and fallbacks

| Risk                                        | Fallback                                                                                   |
|---------------------------------------------|--------------------------------------------------------------------------------------------|
| WhatsApp setup eats the morning             | After 45 minutes stuck, switch to an approve button in the dashboard; keep going           |
| Personas give inconsistent results on stage | Fixed personas and low temperature; pre-test the break so the drop is reliable             |
| Grok rate limits or slow responses          | Cap at 20 personas; run them in parallel; cache the healthy-state baseline                 |
| Shopify API write fails live                | Test the write 3 times before 15:00; keep the before/after visible in the dashboard anyway |
| Everything breaks at 16:25                  | Submit the backup video recorded in Phase 5                                                |

Open questions to settle first: team size, and whether Grok Bot access is through an API key or the hackathon skill (ask in the grok-bot channel).